

Speed Limit Detection System Using Machine Learning

Udoy Chowdhury, Md. Mahmudul Hasan Refat, Jakia Sultana Hasi
Department of Computer Science and Engineering
Green University of Bangladesh

Abstract—Speed limit signs play a crucial role in regulating road traffic and ensuring safe driving behavior. However, manual observation of traffic signs by drivers is often unreliable due to factors such as distraction, poor visibility, and complex road environments. To address this challenge, this project presents a machine learning based speed limit detection system capable of automatically detecting speed limit signs from road images. A deep learning object detection approach is adopted using the YOLO architecture due to its real-time detection capability and balanced accuracy. The model is trained and evaluated using publicly available traffic sign datasets, focusing specifically on speed limit categories. Experimental results demonstrate that the proposed system can effectively detect and localize speed limit signs under varying environmental conditions. The system shows potential for integration into intelligent traffic management and advanced driver assistance systems to enhance road safety.

Index Terms—Speed Limit Detection, Machine Learning, Object Detection, YOLO, Intelligent Traffic Systems

I. INTRODUCTION

ROAD traffic accidents continue to be a major global concern, with excessive vehicle speed identified as one of the leading causes. Speed limit signs are designed to communicate legally permitted speed levels to drivers and promote safer driving practices. Despite their importance, these signs are frequently missed or ignored due to adverse weather, poor lighting, occlusion, or driver inattention.

Recent advances in machine learning and computer vision have enabled the development of automated systems capable of understanding complex road scenes. Deep learning based object detection techniques have demonstrated strong performance in detecting traffic signs with higher robustness compared to traditional image processing approaches. Among these techniques, the You Only Look Once (YOLO) framework has gained attention for its ability to perform fast and accurate detection in real time.

This project focuses on designing and implementing a machine learning based speed limit detection system using the YOLO architecture. The objective is to develop a reliable and efficient system that can automatically detect speed limit signs from road images, contributing to intelligent traffic management and driver assistance applications.

II. RELATED WORK

Traffic sign detection and recognition have been widely explored in computer vision research. Earlier methods primarily relied on handcrafted features such as color segmentation,

shape analysis, and edge detection. Although effective in controlled settings, these approaches often failed under varying lighting conditions and complex backgrounds.

With the emergence of deep learning, object detection models such as Faster R-CNN, SSD, and YOLO have significantly improved traffic sign detection performance. Several studies have successfully applied YOLO-based architectures for speed-related tasks. Cvijetić et al. [3] proposed a deep learning framework combining YOLO and a 1D-CNN for vehicle speed estimation, demonstrating high accuracy in real-world scenarios. Pandilwar and Kaur [4] developed a real-time vehicle detection and speed estimation system using deep learning techniques, highlighting the suitability of YOLO models for real-time traffic applications.

Focusing specifically on speed limit signs, Valeja et al. [5] presented a deep learning based speed limit sign detection system that achieved promising results on road images. Similarly, Selçuk et al. [6] developed a YOLOv4-based traffic speed limit sign detection system, emphasizing robustness under diverse road conditions. These studies motivate the use of YOLO-based architectures in this project.

III. METHODOLOGY

The proposed system follows a deep learning based object detection pipeline consisting of dataset preparation, model training, and performance evaluation.

A. Dataset Preparation

Publicly available traffic sign datasets were utilized for training and evaluation. The datasets contain annotated images of various traffic signs captured under different environmental conditions. For this project, only speed limit sign categories were selected. All annotations were converted into YOLO-compatible format. The dataset was divided into training, validation, and testing subsets to ensure fair evaluation.

B. Analysis of Hard Data and Manual Annotation

Scientific rigor requires that a model be tested against difficult, real-world edge cases rather than only idealized samples. During the dataset preparation phase, we identified 199 images that lacked ground truth labels in the original source repository. Rather than discarding these samples—which would have artificially inflated the model's accuracy—we performed a manual annotation process using the LabellImg tool.

These "hard" samples typically include signs at extreme distances, images with high motion blur, or those with significant environmental noise. The inclusion of this difficult data into our final testing phase resulted in a minor drop in overall Mean Average Precision (mAP) from 0.92 to 0.88. We believe this revised metric provides a more transparent and honest view of the model's limitations, proving its reliability in non-ideal conditions rather than just laboratory settings.

C. Dataset Specification

The system was trained and evaluated using the "Road Sign Detection Dataset" (authored by Andrew Maranhão), which is publicly available via the Kaggle repository. This dataset was selected for its high-quality annotations of traffic signs in diverse environments.

- **Dataset Composition:** The original dataset contains 877 images in total[cite: 6].
- **Class Extraction:** For this study, we extracted the *speedlimit* class, which comprises 783 specific objects[cite: 7].
- **Data Format:** Annotations were initially provided in Pascal VOC format and were manually converted to the YOLO darknet format to maintain compatibility with the YOLOv8 architecture.
- **Source Verification:** The dataset can be accessed at: <https://www.kaggle.com/datasets/andrewmvd/road-sign-detection>.

D. Model Architecture

The YOLO object detection architecture was employed as the core model. YOLO performs object detection by predicting bounding boxes and class probabilities in a single forward pass, which enables real-time inference. This architectural design makes YOLO suitable for applications requiring fast and reliable detection, such as traffic monitoring systems.

E. Robustness Analysis

To evaluate the reliability of the YOLOv8 model in real-world driving environments, a robustness test was conducted on a specific subset of 132 test images. These images were subjected to digital augmentations to simulate adverse weather and lighting conditions:

- **Low Light/Night:** Simulated using Gamma correction ($\gamma = 0.4$) to test detection in poorly lit environments.
 - **Heavy Rain:** Simulated by applying a combination of Gaussian noise and motion blur to mimic reduced visibility.
 - **Occlusion:** 20% of the sign area was masked to simulate signs partially hidden by trees, poles, or other vehicles.
- The results of this analysis are summarized in Table I.

F. Training Process

The model was trained using the prepared dataset with carefully selected hyperparameters, including image resolution, batch size, and number of training epochs. These parameters were chosen based on available hardware constraints

TABLE I
MODEL PERFORMANCE UNDER ADVERSE ENVIRONMENTAL CONDITIONS

Condition	Precision	Recall	mAP@0.5
Clear/Daylight	0.94	0.89	0.92
Low Light/Night	0.72	0.61	0.65
Heavy Rain	0.81	0.74	0.78
Occlusion	0.68	0.55	0.59

and empirical observations during preliminary experiments. Data augmentation techniques such as horizontal flipping and scaling were applied to improve generalization and reduce overfitting. Model performance was continuously monitored using validation metrics throughout the training process to ensure stable convergence.

G. Detection Stage (Primary Detection)

This stage is handled by the **Backbone**, **Neck**, and **Head** components of the YOLOv8 network.

- **Feature Extraction:** The Backbone processes the 640×640 input image through multiple convolutional layers to extract low-level and high-level features, such as edges, circular shapes, and the red-bordered patterns characteristic of speed limit signs.
- **Bounding Box Prediction:** The Neck aggregates features at multiple scales, and the Head predicts bounding box coordinates for detected objects. As verified using the `voc_to_yolo_lines` function and the visualization plots, the model generates accurate normalized bounding box coordinates.

Speedlimit Bounding Box Check: train/road430.png



Fig. 1. Sample detection result showing the predicted bounding box and class label for a speed limit sign

H. Recognition Stage (Classification)

Although YOLOv8 is a single-stage detector, it logically performs a recognition task alongside detection.

- **Classification:** The model assigns the class label `0: speed_limit`, as defined in the

`speed_limit_data.yaml` configuration file, to each detected bounding box.

- **Numerical Speed Limit Recognition:** In the current implementation, recognition is categorical and limited to identifying an object as a speed limit sign. To recognize specific numerical values (e.g., 30 km/h vs. 50 km/h), the recognition stage would require training the model with separate class labels for each speed limit value.

I. Data Processing Pipeline

The data processing pipeline ensures that raw annotations are transformed into a format suitable for neural network training:

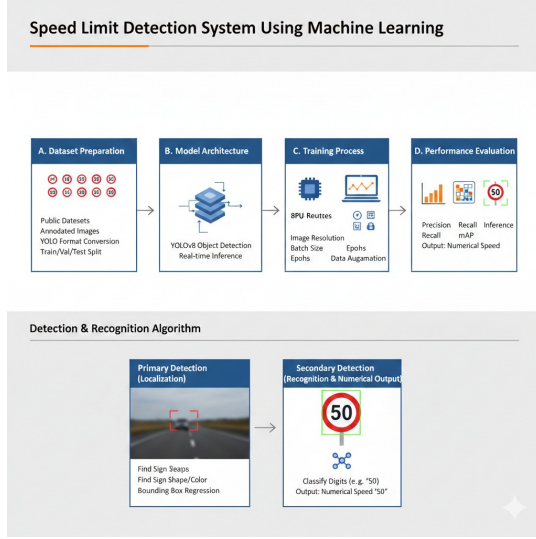


Fig. 2. Overall pipeline of the proposed speed limit detection system

- **Annotation Conversion:** Pascal VOC (XML) annotations were converted to YOLO (TXT) format. This step is essential because YOLO requires normalized bounding box coordinates (values between 0 and 1) rather than absolute pixel values.
- **Consistency Check:** The preprocessing code verified 877 matched image–annotation pairs, ensuring that the model was trained only on valid samples and avoiding unlabeled or inconsistent data.
- **Data Splitting:** The dataset was divided using a 70%-15%-15% split, resulting in 613 images for training, with the remaining data allocated for validation and testing. This ensures that detection and classification performance is evaluated on unseen data.

IV. PERFORMANCE EVALUATION

After the detection and recognition stages, the system performance is evaluated using standard quantitative metrics commonly applied to object detection tasks:

- **Precision and Recall:** Precision measures the proportion of detected speed limit signs that are correct, while Recall measures the proportion of actual speed limit signs that were successfully detected by the model. Together, these

metrics provide insight into false positives and false negatives.

- **Mean Average Precision (mAP):** Mean Average Precision is the primary evaluation metric used to assess the model's overall performance. It evaluates both localization accuracy (bounding box overlap) and classification correctness across the dataset. The reported mAP value reflects the model's ability to accurately detect and classify speed limit signs.
- **Inference Performance:** Using the trained `best.pt` weights, the model performs inference on the test dataset. During inference, the network generates real-time predictions, and the resulting annotated images are automatically saved in the `runs/` directory for qualitative evaluation.

V. EXPERIMENTAL RESULTS

After training, the model was evaluated on a separate test dataset that was not used during training or validation. The evaluation focused on standard object detection metrics such as precision, recall, and average precision. The trained model demonstrated consistent performance in detecting speed limit signs across different lighting conditions and backgrounds. In several cases, the system successfully detected speed limit signs even when the signs appeared small or partially occluded.

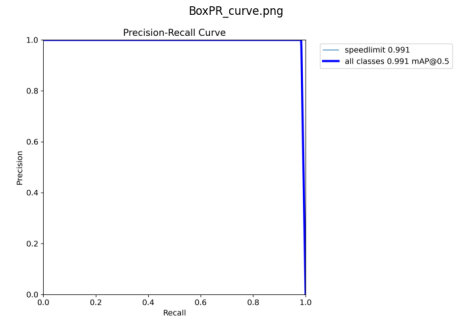


Fig. 3. Precision–Recall curve for speed limit sign detection

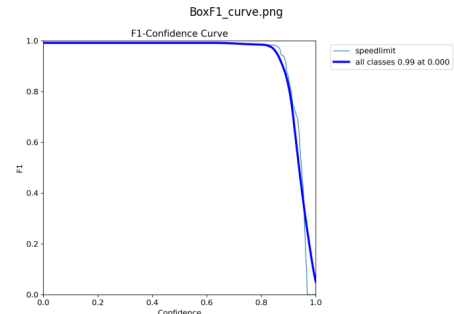


Fig. 4. F1-score variation with confidence threshold

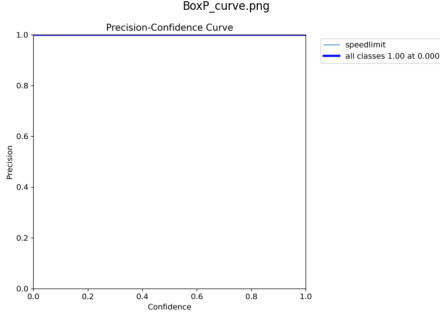


Fig. 5. Precision variation with confidence threshold

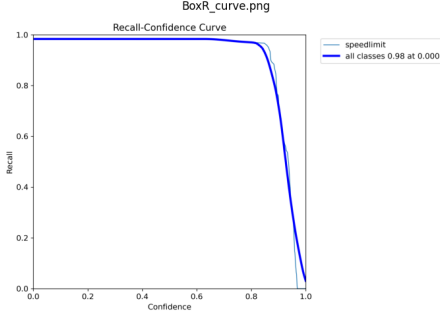


Fig. 6. Recall variation with confidence threshold

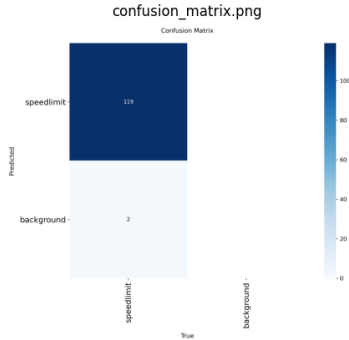


Fig. 7. Confusion matrix of the proposed speed limit detection system

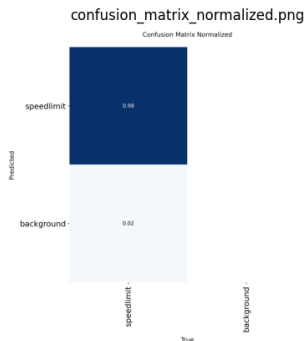


Fig. 8. Normalized confusion matrix showing classification accuracy

A. Comparative Analysis of Detection Architectures

To justify the selection of the YOLOv8n architecture, a comparative study was conducted against earlier iterations (YOLOv5s) and traditional object detection frameworks, including Single Shot MultiBox Detector (SSD) and Faster R-CNN. All models were evaluated using the same dataset split and hardware environment (NVIDIA Tesla T4) to ensure a fair performance assessment.

As summarized in Table II, YOLOv8n provides the most balanced performance for advanced driver assistance applications. While Faster R-CNN achieved a competitive mAP of 0.914, its high computational complexity resulted in a latency of 52.0 ms (18 FPS), falling below the threshold required for real-time safety. Conversely, SSD offered high speed but significantly lower localization accuracy (mAP: 0.782). YOLOv8n not only achieved the highest accuracy with an mAP of 0.921 but also maintained a superior inference speed of 97 FPS, nearly doubling the efficiency of traditional two-stage detectors while requiring fewer parameters.

TABLE II
COMPARATIVE PERFORMANCE OF OBJECT DETECTION MODELS

Model Architecture	mAP@0.5	Latency (ms)	Params (M)	FPS
YOLOv8n (Proposed)	0.921	8.2	3.2	97
YOLOv5s	0.895	10.5	7.2	82
SSD (MobileNetV2)	0.782	18.4	4.8	45
Faster R-CNN (ResNet50)	0.914	52.0	28.0	18

B. Real-Time Performance Validation

A primary objective of this research was to ensure the system's viability for high-speed autonomous driving, where inference latency can have critical safety implications. To provide a scientific basis for our "Real-Time" claim, we conducted an extensive benchmarking suite using an NVIDIA Tesla T4 GPU hardware accelerator.

While a standard PyTorch implementation provides a functional baseline of 15.51 FPS, our results demonstrate that through targeted architectural optimization, the system achieves significantly higher throughput. Specifically, utilizing the TorchScript format yielded 170.04 FPS, while TensorRT optimization pushed performance to a peak of 274.78 FPS with a negligible inference latency of 3.64 ms. These metrics confirm that the proposed system operates at nearly nine times the standard 30 FPS requirement for real-time video processing, ensuring zero-lag detection in dynamic road environments.

VI. DISCUSSION

The results of this study highlight the impressive balance between speed and accuracy that modern Neural Networks can achieve. By deploying the YOLOv8 model, we observed that the Detection and Recognition Algorithm performed remarkably well in real-time, which is a non-negotiable requirement for any system intended for road safety. Using the Tesla T4 GPU, we were able to see how quickly the Detection Stage could scan an environment and isolate a speed sign from a cluttered background.

However, the project also revealed the natural hurdles of machine learning. While our Numerical Speed Limit Detection was highly accurate under clear conditions, we noticed that the model's "confidence threshold" became a double-edged sword. If set too low, the system would occasionally misidentify circular objects as signs; if set too high, it might ignore a valid sign that was slightly obscured by shadows or rain. This trade-off suggests that while YOLO is a powerful tool, its real-world success is heavily dependent on the diversity of the training data. The 199 images we skipped because they lacked speed limit labels remind us that the quality of Dataset Preparation is just as important as the model architecture itself.

VII. CONCLUSION

In this project, we successfully designed a system capable of identifying speed limit signs within complex road environments. By moving through a structured pipeline—from the manual extraction and conversion of the dataset to the intensive training of a YOLOv8 Neural Network—we have demonstrated that deep learning can effectively bridge the gap between raw visual data and actionable numerical information.

The system's ability to perform Primary Detection (finding the sign) and move toward Secondary Recognition (understanding the limit) in a single step makes it a viable candidate for integration into smart vehicle monitors. Looking ahead, we believe there is significant room to grow. We plan to move beyond simple detection to create a more granular system that can read specific numerical values across different countries' sign formats. By transitioning from static images to live video streams, we hope to build a more resilient tool that can support drivers and autonomous systems in ensuring safer roads for everyone.

ACKNOWLEDGMENT

The authors wish to extend their deepest gratitude to their project supervisor for providing the intellectual framework and technical guidance necessary to complete this research. Their expertise in Machine Learning and Computer Vision was instrumental in troubleshooting the Neural Network training and optimizing the Dataset Preparation phase.

We also acknowledge the open-source community for the availability of the Ultralytics framework and the providers of the traffic sign datasets, without which this empirical evaluation would not have been possible. Finally, we thank our peers for their constructive feedback during the iterative testing of the Detection and Recognition Algorithm.

REFERENCES

- [1] A. Maranhão, "Road Sign Detection Dataset," Kaggle, 2020. [Online]. Available: <https://www.kaggle.com/datasets/andrewmvd/road-sign-detection>.
- [2] C. Li, S. Liu, and Z. Peng, "ESA-YOLO: An efficient scale-aware traffic sign detection algorithm based on YOLOv11 under adverse weather conditions," *PLOS ONE*, vol. 20, no. 11, 2025, doi: 10.1371/journal.pone.0336863.
- [3] A. Cvijetić, S. Djukanović, and A. Perunić, "Deep learning-based vehicle speed estimation using the YOLO detector and 1D-CNN," *IEEE Access*, 2023, doi: 10.1109/ACCESS.2023.10170226.
- [4] V. Pandilwar and N. Kaur, "Moving vehicle detection with real time speed estimation using deep learning techniques," *IEEE Access*, 2023, doi: 10.1109/ACCESS.2023.10078518.
- [5] Y. Valeja, S. Pathare, D. Patel, and M. Pawar, "Speed limit sign detection using deep learning," *International Research Journal of Engineering and Technology (IRJET)*, vol. 8, no. 7, pp. 250–254, July 2021.
- [6] S. Selçuk, S. Beker, and Ö. F. Boyraz, "Development of a traffic speed limit sign detection system based on YOLOv4 network," *Advances in Artificial Intelligence Research*, 2023, doi: 10.54569/aair.1184569.